

Congressional Voting Records - Zadanie projektowe

Opis problemu

Celem projektu jest zbudowanie modelu klasyfikacyjnego, który na podstawie zapisu głosowań przewiduje przynależność partyjną członka Izby Reprezentantów USA (demokrata / republikanin). Zbiór danych zawiera wyłącznie zmienne kategoryczne, co wymaga odpowiedniego kodowania przed modelowaniem.

Zbiór dotyczy 16 kluczowych głosowań z 1984 roku. Analiza ma charakter wyłącznie techniczny — celem jest ćwiczenie metod klasyfikacji i interpretacji cech, a nie formułowanie ocen politycznych.

Zbiór danych

- Źródło: [UCI Machine Learning Repository](#) (licencja CC BY 4.0)
- Plik: `house-votes-84.data`
- Rozmiar: **435 rekordów, 17 kolumn** (1 zmienna celu + 16 cech)

Struktura danych

Plik nie zawiera nagłówek - nazwy kolumn należy przypisać z listy `COLUMN_NAMES`. Każda cecha głosowania przyjmuje wartości: `y` (za), `n` (przeciw), `?` (pozycja nieujawniona).

#	Kolumna	Wartości
0	class	democrat, republican (ZMIENNA CELU)
1	handicapped-infants	y, n, ?
2	water-project-cost-sharing	y, n, ?
3	adoption-of-the-budget-resolution	y, n, ?
4	physician-fee-freeze	y, n, ?
5	el-salvador-aid	y, n, ?
6	religious-groups-in-schools	y, n, ?
7	anti-satellite-test-ban	y, n, ?
8	aid-to-nicaraguan-contras	y, n, ?
9	mx-missile	y, n, ?
10	immigration	y, n, ?
11	synfuels-corporation-cutback	y, n, ?
12	education-spending	y, n, ?
13	superfund-right-to-sue	y, n, ?
14	crime	y, n, ?

#	Kolumna	Wartości
15	duty-free-exports	y, n, ?
16	export-administration-act-south-africa	y, n, ?

Brakujące wartości

Symbol ? występuje we **wszystkich 16 kolumnach** głosowań (łącznie ok. 390 wystąpień, 203 wiersze zawierają co najmniej jeden ?). **Nie jest to klasyczny brak danych** - oznacza pozycję nieujawnioną (np. wstrzymanie się od głosu). Należy świadomie wybrać strategię jego obsługi.

Wymagania formalne

Forma rozwiązania

Rozwiązanie składa się z **dwóch plików**:

1. **solution.py** - moduł Python z implementacją wszystkich wymaganych funkcji
2. **wnioski.md** - dokument Markdown z interpretacją wyników i wnioskami

Wymagania dla solution.py

1. Użyj dostarczonego szablonu **solution_template.py**
2. **Nie zmieniaj** nazw funkcji, parametrów ani typów zwracanych
3. Zaimplementuj **wszystkie** funkcje oznaczone **# TODO**
4. Możesz dodawać własne funkcje pomocnicze (nazwy zaczynające się od **_**)
5. Kod musi być uruchamialny bez błędów
6. Funkcja **run_full_analysis()** musi zwracać kompletny słownik wyników

Wymagania dla wnioski.md

Dokument powinien zawierać następujące sekcje (użyj pliku **wnioski_template.md**):

1. **Analiza danych** (max 200 słów)
 - Rozkład klas (czy zbalansowany?)
 - Obsługa wartości ? - jaką strategię wybrałeś i dlaczego?
 - Najsilniejsze zależności (Cramér's V)

2. **Porównanie modeli** (max 200 słów)

- Który model osiągnął najlepsze wyniki?
- Porównanie metryk (Accuracy, Precision, Recall, F1, AUC)
- Czy NaiveBayes spisał się dobrze na danych kategoriowych?

3. **Analiza błędów** (max 150 słów)

- Ile było błędów False Positive i False Negative?
- Czy błędy rozkładają się symetrycznie między klasami?
- W jakim zastosowaniu asymetria kosztu błędu (Precision vs Recall) miałaby znaczenie?

4. **Interpretacja cech** (max 150 słów)

- Top 5 najważniejszych cech według RandomForest
- Czy pokrywają się z rankingiem Craméra?
- Które głosowania najsilniej różnicują obie grupy?

5. **Wnioski** (max 100 słów)

- Podsumowanie projektu
- Czy zapis głosowań pozwala wiarygodnie przewidywać przynależność?

Wymagane załączniki (obrazki):

- Rozkład klas
- Wykres najsilniejszej cechy (**physician-fee-freeze**) względem klasy
- Wykres drugiej cechy (**adoption-of-the-budget-resolution**) względem klasy
- Ranking Cramér's V
- Macierz pomyłek
- Krzywa ROC
- Ważność cech (RandomForest)

Specyfikacja funkcji

Część 1: Wczytanie i eksploracja danych

```
load_data(filepath: str) -> pd.DataFrame
```

Wczytuje dane i przypisuje nazwy kolumn z COLUMN_NAMES.

```
get_basic_info(df: pd.DataFrame) -> Dict[str, Any]
```

Zwraca: 'shape', 'dtypes', 'missing_values', 'missing_question_marks'

```
get_class_distribution(df: pd.DataFrame) -> Dict[str, Any]
```

Zwraca: 'counts', 'percentages', 'is_balanced'

```
get_feature_info(df: pd.DataFrame, feature: str) -> Dict[str, Any]
```

Zwraca: 'unique_values', 'unique_count', 'value_counts', 'missing_count'

Część 2: Analiza eksploracyjna

```
calculate_cramers_v(x: pd.Series, y: pd.Series) -> float
```

Oblicza współczynnik V Craméra.

```
get_cramers_v_for_features(df: pd.DataFrame, features: List[str] = None) ->  
pd.DataFrame
```

Zwraca DataFrame (feature, cramers_v) posortowany malejąco.

```
get_feature_class_crosstab(df: pd.DataFrame, feature: str) -> pd.DataFrame
```

Zwraca tabelę krzyżową z normalizacją po kolumnach.

Część 3: Przygotowanie danych

```
handle_missing_values(df: pd.DataFrame, strategy: str = 'keep_as_category')  
-> pd.DataFrame
```

Obsługuje wartości ? ('keep_as_category', 'drop_rows', 'impute_mode').

```
prepare_features(df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]
```

Zwraca (X, y) gdzie y: democrat=0, republican=1.

`encode_features(X: pd.DataFrame, method: str = 'onehot') -> np.ndarray`

Koduje cechy ('onehot' lub 'ordinal').

`get_feature_names_after_encoding(X: pd.DataFrame) -> List[str]`

Zwraca nazwy cech po OneHot encoding.

`split_data(X, y, test_size=0.2, random_state=42) -> Tuple[...]`

Dzieli dane ze stratyfikacją.

Część 4: Budowa modeli

`get_models() -> Dict[str, Any]`

Zwraca słownik z modelami:

- 'LogisticRegression': LogisticRegression(max_iter=1000, random_state=42)
- 'DecisionTree': DecisionTreeClassifier(max_depth=4, random_state=42)
- 'RandomForest': RandomForestClassifier(n_estimators=50, max_depth=5, random_state=42)
- 'GradientBoosting': GradientBoostingClassifier(n_estimators=50, learning_rate=0.1, max_depth=2, random_state=42)
- 'NaiveBayes': CategoricalNB()

UWAGA: Hiperparametry celowo ograniczone, aby pokazać różnice między modelami.

`train_model(model, X_train, y_train) -> Any`

Trenuje i zwraca model.

Część 5: Ocena modeli

`calculate_metrics(y_true, y_pred, y_prob=None) -> Dict[str, float]`

Zwraca: 'accuracy', 'precision', 'recall', 'f1', 'auc'

`evaluate_all_models(...)` -> `pd.DataFrame`

Zwraca DataFrame z wynikami wszystkich modeli (Model, Accuracy, Precision, Recall, F1, AUC).

`get_best_model_name(results_df, metric='F1')` -> `str`

Zwraca nazwę najlepszego modelu.

`get_confusion_matrix(y_true, y_pred)` -> `np.ndarray`

Zwraca macierz pomyłek 2x2.

`get_roc_curve_data(y_true, y_prob)` -> `Dict[str, Any]`

Zwraca: 'fpr', 'tpr', 'thresholds', 'auc'

Część 6: Interpretacja modelu

`get_feature_importance(model, feature_names)` -> `pd.DataFrame`

Zwraca DataFrame (feature, importance) posortowany malejąco.

`get_top_features(importance_df, n=5)` -> `List[str]`

Zwraca listę n najważniejszych cech.

`analyze_misclassification_risk(cm: np.ndarray)` -> `Dict[str, Any]`

Zwraca: 'false_positives', 'false_negatives', 'fn_rate', 'fp_rate', 'error_assessment'

Część 7: Wizualizacje i funkcja główna

Wszystkie funkcje `plot_*` zwracają `plt.Figure`.

```
generate_all_figures(df, results, output_dir="figures") -> Dict[str, str]
```

Generuje i zapisuje wszystkie wykresy.

```
run_full_analysis(filepath, generate_figures=True, figures_dir="figures") ->  
Dict[str, Any]
```

Uruchamia pełną analizę.

System oceniania

Etap	Opis	Punkty
1. Wczytanie i eksploracja	<code>load_data</code> , <code>get_basic_info</code> , <code>get_class_distribution</code> , <code>get_feature_info</code>	10 pkt
2. Analiza eksploracyjna	<code>calculate_cramers_v</code> , <code>get_cramers_v_for_features</code> , <code>get_feature_class_crosstab</code>	10 pkt
3. Przygotowanie danych	<code>handle_missing_values</code> , <code>prepare_features</code> , <code>encode_features</code> , <code>get_feature_names_after_encoding</code> , <code>split_data</code>	10 pkt
4. Budowa modeli	<code>get_models</code> , <code>train_model</code>	10 pkt
5. Ocena modeli	<code>calculate_metrics</code> , <code>evaluate_all_models</code> , <code>get_best_model_name</code> , <code>get_confusion_matrix</code> , <code>get_roc_curve_data</code>	15 pkt
6. Interpretacja	<code>get_feature_importance</code> , <code>get_top_features</code> , <code>analyze_misclassification_risk</code>	10 pkt
7. Wizualizacje	Wszystkie funkcje <code>plot_*</code> i <code>generate_all_figures</code>	15 pkt
8. Integracja	<code>run_full_analysis</code>	5 pkt
9. Wnioski.md	Dokument z interpretacją i obrazkami	15 pkt

Łącznie: 100 punktów

Materiały pomocnicze

- [Scikit-learn dokumentacja](#)
- [Cramér's V - Wikipedia](#)
- [Confusion Matrix interpretation](#)